

Interface Specification ver_2.0

Just Pick It Interface Specification

1. 작성 기준

이 문서는 “누가 누구에게 무엇을 보내고, 무엇을 받는가”를 기준으로 인터페이스를 분류한다.

항목	기준
UI 통신	Customer UI와 Admin UI는 Web Gateway의 HTTP/WebSocket API를 사용한다.
Web Gateway	Web Gateway는 화면 렌더링과 <code>/api/*</code> 프록시를 담당한다. DB에 직접 접근하지 않는다.
Fleet API	Fleet API는 Fleet Manager 프로세스 내부에서 실행되는 HTTP/WebSocket API이다.
데이터 저장	DB 읽기/쓰기는 Fleet Manager의 FleetRepository가 담당한다. Web Gateway, UI, 로봇은 DB에 직접 접근하지 않는다.
주문 감지	TaskManager가 FleetRepository를 통해 ORDER_WAIT 주문을 주기적으로 확인한다.
진열 감지	TaskManager가 FleetRepository를 통해 REQUESTED display_item을 주기적으로 확인한다.
주문 task 생성/배정	TaskManager가 task 생성, robot_unit 배정, sequence_no 부여를 담당한다.
진열 task 생성/배정	TaskManager가 display_item을 읽어 진열 task 5개를 생성한다.

경로/교통 판단	TrafficManager가 PICKY 이동 경로와 구역 점유 가능 여부를 판단한다.
로봇 제어	PICKY와 COBOT은 Web Gateway와 직접 task를 주고받지 않고 Fleet Manager를 통해 제어된다.
로봇 상태 보고	State Manager가 ROS2 Topic으로 상태를 발행하고, RobotStateMonitor가 수신해 DB에 반영한다.
LLM	Web Gateway가 LLM 담당 모듈을 호출한다. DISPLAY로 파싱되면 Fleet API에 display_item 생성을 요청한다.
Vision	PICKY/COBOT State Manager 또는 Vision Node가 AI Vision Server에 직접 요청한다.
이미지/영상	Web Gateway와 Fleet API는 이미지/영상 프레임을 중계하지 않는다.
실시간 UI	Fleet API가 Customer/Admin UI 상태를 WebSocket으로 push하고, Web Gateway가 이를 프록시한다.
ROS2 통신	Fleet Manager와 State Manager가 ROS2 Topic/Action/Service로 PICKY/COBOT을 제어한다.

2. 전체 흐름 요약

구분	흐름	핵심 역할
고객 주문	Customer UI → Web Gateway → Fleet API	상품 조회, 주문 생성, 주문 상태 확인, 수령 완료
관리자 관제	Admin UI → Web Gateway → Fleet API	로봇/주문/task/재고/픽업슬롯/예외 상태 조회 및 제어

주문 감지	TaskManager → FleetRepository → DB	ORDER_WAIT 주문 polling
주문 작업 생성	TaskManager → FleetRepository → DB	task 생성, robot_unit 배 정, 상태 전이
진열 명령 생성	Admin UI → Web Gateway → LLM → Fleet API	자연어 진열 명령 파싱 후 display_item 생성
진열 작업 감지	TaskManager → FleetRepository → DB	REQUESTED display_item polling
진열 작업 생성	TaskManager → FleetRepository → DB	진열 task 생성, display_item 상태 변경, 진열 완료 처리
로봇 제어	Fleet Manager → PICKY/COBOT State Manager	이동, 상차, 검수, 하차, 진열, 복귀, 도킹, 충전, 긴급정지 명령
Vision 인식	PICKY/COBOT State Manager → AI Vision Server	상품 인식, 검수, 진열 상 품 인식, 사람 감지
UI 실시간 갱신	Fleet API → Web Gateway → Customer/Admin UI	WebSocket 상태 push

3. Customer UI ↔ Web Gateway / Fleet API

고객이 상품을 조회하고 주문하며, 진행 상태를 확인하고 수령 완료까지 처리하는 흐름이다. 브라우저는 같은 origin의 Web Gateway를 호출하고, Web Gateway가 Fleet API로 프록시한다.

IF ID	호출 시 점	송신자	수신자	목적	요청	응답 형 태	프로토 콜
IF-C-01	고객이 상품 목 록 화면 진입 시	Custo mer UI	Web Gatewa y →	상품 목 록 조회	GET /api/pr oducts	product 목록	HTTP/ TCP

			Fleet API				
IF-C-02	고객이 상품 선택/수량 변경 시	Customer UI	Customer UI	장바구니 상태 변경	서버 API 없음	브라우저 local state	Local
IF-C-03	고객이 주문 확정 시	Customer UI	Web Gateway → Fleet API	주문 생성	POST /api/orders	order 상세	HTTP/TCP
IF-C-04	고객이 주문 현황 확인 시	Customer UI	Web Gateway → Fleet API	주문 상태 조회	GET /api/orders/{order_id}	order 상세	HTTP/TCP
IF-C-05	고객 화면 초기 진입/새로고침 시	Customer UI	Web Gateway → Fleet API	고객 화면 전체 상태 조회	GET /api/customer/status	products, orders	HTTP/TCP
IF-C-06	주문/상품 상태 변경 시 자동	Fleet API → Web Gateway	Customer UI	고객 화면 실시간 상태 push	WS /api/customer/ws/status	customer status	WebSocket/TCP
IF-C-07	고객이 상품 수령 후	Customer UI	Web Gateway → Fleet API	수령 완료 처리	POST /api/orders/{order_id}/complete	completed order	HTTP/TCP

```
1 {
2   "items": [
3     {
4       "product_id": 1,
5       "quantity": 2
6     },
7     {
8       "product_id": 3,
9       "quantity": 1
10    }
11  ]
12 }
```

IF-C-03 처리 원칙

항목	처리
orders	생성 후 status=ORDER_WAIT
order_item	주문 상품 목록 생성
product	주문 수량만큼 stock_qty 차감
task	주문 생성 시점에는 생성하지 않음
Fleet Manager	이후 TaskManager가 ORDER_WAIT 주문을 감지해 task 생성

4. Admin UI ↔ Web Gateway / Fleet API

관리자가 로봇, 주문, 작업, 재고, 픽업 슬롯, 예외를 조회하고 제어하는 흐름이다.

IF ID	호출 시 점	송신자	수신자	목적	요청	응답 형 태	프로토 콜
IF-A-01	관리자 대시보 드 진입 시	Admin UI	Web Gatewa y → Fleet API	관리자 통합 상 태 조회	GET /api/ad min/sta tus	admin status	HTTP/ TCP
IF-A-02	상태 변 경 시 자 동	Fleet API → Web Gatewa y	Admin UI	관리자 화면 실 시간 상 태 push	WS /api/ad min/ws /status	admin status	WebSo cket/T CP

IF-A-03	관리자가 긴급 정지 클릭 시	Admin UI	Web Gateway → Fleet API	전체 로봇 긴급 정지 요청	POST /api/admin/emergency-stop	{status: "ok"}	HTTP/TCP
IF-A-04	관리자가 재개 클릭 시	Admin UI	Web Gateway → Fleet API	긴급 정지 해제/재개 요청	POST /api/admin/resume	{status: "ok"}	HTTP/TCP
IF-A-05	관리자가 신규 상품 등록 시	Admin UI	Web Gateway → Fleet API	상품 등록	POST /api/admin/products	product	HTTP/TCP
IF-A-06	관리자가 상품 정보 수정 시	Admin UI	Web Gateway → Fleet API	상품 수정	PATCH /api/admin/products/{product_id}	product	HTTP/TCP
IF-A-07	관리자가 재고 수량만 수정 시	Admin UI	Web Gateway → Fleet API	재고 수량 수정	PATCH /api/admin/products/{product_id}/stock	product	HTTP/TCP
IF-A-08	관리자가 자연어 진열 명령 입력 시	Admin UI	Web Gateway → LLM → Fleet API	LLM 진열 명령 처리	POST /api/admin/llm/messages	LLM 처리 결과 및 display_item	HTTP/TCP

IF-A-09	관리자가 예외 처리 완료 시	Admin UI	Web Gateway → Fleet API	예외 resolved 처리	POST /api/admin/exceptions/{exception_id}/resolve	{status: "ok"}	HTTP/TCP
IF-A-10	관리자가 픽업 슬롯 상태 변경 시	Admin UI	Web Gateway → Fleet API	픽업 슬롯 상태 수동 변경	PATCH /api/fleet/pickup-slots/{slot_id}	{status: "ok"}	HTTP/TCP
IF-A-11	관리자가 task 수동 보정 시	Admin UI	Web Gateway → Fleet API	task 상태 수동 변경	PATCH /api/fleet/tasks/{task_id}	{status: "ok"}	HTTP/TCP
IF-A-12	관리자가 robot 수동 보정 시	Admin UI	Web Gateway → Fleet API	robot 상태 수동 보정	PATCH /api/fleet/robots/{robot_id 또는 robot_name}	{status: "ok"}	HTTP/TCP

IF-A-08 요청 예시

```

1 {
2   "message": "콜라 5개 진열해줘"
3 }
```

IF-A-08 응답 예시

```

1 {
2   "result": "ok",
3   "message": "진열 요청이 생성되었습니다. display_item_id=1, 상품=콜라",
4   "action": "DISPLAY",
```

```

5  "product_id": 1,
6  "product_name": "콜라",
7  "requested_quantity": 5,
8  "display_policy": "REQUESTED_QUANTITY",
9  "display_item_id": 1,
10 "display_status": "REQUESTED",
11 "provider": "llm"
12 }

```

5. Web Gateway / Fleet API / FleetRepository 경계

기존 Control Server 역할은 분리되었다.

Web Gateway는 DB를 직접 만지지 않고, Fleet API 앞단의 화면 서버/프록시로 동작한다.

Fleet Manager 내부의 FleetRepository가 DB 읽기/쓰기를 담당한다.

5-1. Fleet API 조회 API

IF ID	호출 시 점	송신자	수신자	목적	요청	응답 형 태	프로토 콜
IF-F-01	Web Gateway 또는 관리자 조회	Web Gateway	Fleet API	전체 snapshot 조회	GET /api/fleet/snapshot	admin status 와 유사	HTTP/TCP
IF-F-02	Web Gateway 또는 관리자 조회	Web Gateway	Fleet API	zone 좌표 조회	GET /api/fleet/zones?zone_type={zone_type}	zone 목록	HTTP/TCP
IF-F-03	Web Gateway 또는 관리자 조회	Web Gateway	Fleet API	task 목록 조회	GET /api/fleet/tasks	task 목록	HTTP/TCP

IF-F-04	Web Gateway 또는 관리자 조회	Web Gateway	Fleet API	주문 목록 조회	GET /api/fleet/orders?status={status}	order 목록	HTTP/TCP
IF-F-05	Web Gateway 또는 관리자 조회	Web Gateway	Fleet API	주문별 task 조회	GET /api/fleet/orders/{order_id}/tasks	task 목록	HTTP/TCP
IF-F-06	Web Gateway 또는 관리자 조회	Web Gateway	Fleet API	픽업 슬롯 조회	GET /api/fleet/pickup-slots?status={status}	pickup slot 목록	HTTP/TCP
IF-F-07	고객/관리자 조회	Web Gateway	Fleet API	상품 목록 조회	GET /api/products	product 목록	HTTP/TCP
IF-F-08	고객/관리자 조회	Web Gateway	Fleet API	주문 상세 조회	GET /api/orders/{order_id}	order 상세	HTTP/TCP

5-2. Fleet API 변경 API

IF ID	호출 시점	송신자	수신자	목적	요청	응답 형태	프로토콜
-------	-------	-----	-----	----	----	-------	------

IF-F-09	고객 주문 확정 시	Web Gateway	Fleet API	주문 생성	POST /api/orders	order	HTTP/TCP
IF-F-10	고객 수령 완료 시	Web Gateway	Fleet API	주문 완료	POST /api/orders/{order_id}/complete	order	HTTP/TCP
IF-F-11	관리자 상품 등록 시	Web Gateway	Fleet API	상품 생성	POST /api/admin/products	product	HTTP/TCP
IF-F-12	관리자 상품 수정 시	Web Gateway	Fleet API	상품 수정	PATCH /api/admin/products/{product_id}	product	HTTP/TCP
IF-F-13	관리자 재고 수정 시	Web Gateway	Fleet API	재고 수정	PATCH /api/admin/products/{product_id}/stock	product	HTTP/TCP
IF-F-14	관리자 픽업 슬롯 생성 시	Web Gateway	Fleet API	픽업 슬롯 생성	POST /api/admin/pickup-slots	pickup_slot	HTTP/TCP
IF-F-15	LLM 진열 요청 생성 시	Web Gateway	Fleet API	display_item 생성	POST /api/admin/display	display_item	HTTP/TCP

					play-items		
IF-F-16	관리자 수동 task 생성 시	Web Gateway	Fleet API	task 일 괄 생성	POST /api/fleet/tasks/bulk	task_ids	HTTP/ TCP
IF-F-17	관리자 수동 task 보 정 시	Web Gateway	Fleet API	task 상 태 수정	PATCH /api/fleet/tasks/{task_id}	{status: "ok"}	HTTP/ TCP
IF-F-18	관리자 수동 order 보정 시	Web Gateway	Fleet API	order 상 태/slot/ unit 수 정	PATCH /api/fleet/orders/{order_id}	{status: "ok"}	HTTP/ TCP
IF-F-19	관리자 수동 robot 보정 시	Web Gateway	Fleet API	robot 상태 수 정	PATCH /api/fleet/robots/{robot_id 또는 robot_name}	{status: "ok"}	HTTP/ TCP
IF-F-20	관리자 수동 pickup slot 보 정 시	Web Gateway	Fleet API	pickup slot 상 태 수정	PATCH /api/fleet/pickup-slots/{slot_id}	{status: "ok"}	HTTP/ TCP
IF-F-21	관리자 예외 resolve	Web Gateway	Fleet API	예외 해 결	POST /api/admin/exception	{status: "ok"}	HTTP/ TCP

	d 처리 시				s/{exce ption_i d}/reso lve	
--	-----------	--	--	--	--------------------------------------	--

5-3. TaskManager 내부 polling 처리 흐름

TaskManager는 HTTP로 Fleet API를 호출하지 않는다.

같은 프로세스 안에서 FleetRepository를 통해 DB를 읽고 쓴다.

주문 polling 처리 흐름

```

1 1. TaskManager가 주기적으로 FleetRepository로 ORDER_WAIT 주문을 조회한다.
2 2. 기존 task 존재 여부를 확인한다.
3 3. 기존 task가 있으면 skip한다.
4 4. 기존 task가 없으면 robot_unit을 배정한다.
5 5. 주문 item을 product zone/slot 기준 dict로 정규화한다.
6 6. TrafficManager에 상품 후보 dict를 전달한다.
7 7. TrafficManager가 가장 가까운 상품 zone 경로를 예약한다.
8 8. TaskManager가 선택된 상품에 대한 task를 생성한다.
9 9. task 상태 변화는 FleetRepository를 통해 DB에 반영된다.
10 10. Fleet API WebSocket이 최신 상태를 UI에 push한다.

```

진열 polling 처리 흐름

```

1 1. Admin UI가 자연어 진열 명령을 입력한다.
2 2. Web Gateway LLM parser가 DISPLAY로 파싱하면 Fleet API에 display_item
   생성을 요청한다.
3 3. FleetRepository가 display_item을 REQUESTED로 저장한다.
4 4. TaskManager가 주기적으로 REQUESTED display_item을 조회한다.
5 5. TaskManager가 robot_unit을 배정한다.
6 6. TaskManager가 진열 task 5개를 생성한다.
7 7. DISPLAY_PLACE SUCCESS 시 product.stock_qty와 display_item 상태를 반영
   한다.
8 8. Fleet API WebSocket이 최신 상태를 UI에 push한다.

```

task 일괄 생성 요청 예시

```

1 {
2   "tasks": [
3     {
4       "order_id": 7,
5       "order_item_id": 22,
6       "display_item_id": null,
7       "sequence_no": 1,
8       "assigned_robot_name": "PICKY1",
9       "task_type": "MOVE_TO_PRODUCT",
10      "status": "ASSIGNED",
11      "priority": 2,
12      "source_zone_id": 1,
13      "target_zone_id": 6,
14      "result_message": null
15    },
16    {
17      "order_id": 7,
18      "order_item_id": 22,
19      "display_item_id": null,
20      "sequence_no": 2,
21      "assigned_robot_name": "COBOT1",

```

```

22     "task_type": "SORTING_AND_LOAD",
23     "status": "ASSIGNED",
24     "priority": 2,
25     "source_zone_id": 12,
26     "target_zone_id": 6,
27     "result_message": null
28   }
29 ]
30 }

```

진열 task 생성 예시

```

1  {
2    "tasks": [
3      {
4        "order_id": null,
5        "order_item_id": null,
6        "display_item_id": 1,
7        "sequence_no": 1,
8        "assigned_robot_name": "PICKY1",
9        "task_type": "MOVE_TO_STOCK",
10       "status": "ASSIGNED",
11       "priority": 1,
12       "source_zone_id": 1,
13       "target_zone_id": 3,
14       "result_message": null
15     },
16     {
17       "order_id": null,
18       "order_item_id": null,
19       "display_item_id": 1,
20       "sequence_no": 2,
21       "assigned_robot_name": "COBOT1",
22       "task_type": "SORTING_AND_LOAD",
23       "status": "ASSIGNED",
24       "priority": 1,
25       "source_zone_id": 4,
26       "target_zone_id": 4,
27       "result_message": null
28     },
29     {
30       "order_id": null,
31       "order_item_id": null,
32       "display_item_id": 1,
33       "sequence_no": 3,
34       "assigned_robot_name": "PICKY1",
35       "task_type": "MOVE_TO_DISPLAY",
36       "status": "ASSIGNED",
37       "priority": 1,
38       "source_zone_id": 3,
39       "target_zone_id": 8,
40       "result_message": null
41     },
42     {
43       "order_id": null,
44       "order_item_id": null,
45       "display_item_id": 1,
46       "sequence_no": 4,
47       "assigned_robot_name": "COBOT1",
48       "task_type": "DISPLAY_SCAN",
49       "status": "ASSIGNED",
50       "priority": 1,
51       "source_zone_id": 14,
52       "target_zone_id": 14,
53       "result_message": null
54     },

```

```

55     {
56         "order_id": null,
57         "order_item_id": null,
58         "display_item_id": 1,
59         "sequence_no": 5,
60         "assigned_robot_name": "COBOT1",
61         "task_type": "DISPLAY_PLACE",
62         "status": "ASSIGNED",
63         "priority": 1,
64         "source_zone_id": 14,
65         "target_zone_id": 14,
66         "result_message": null
67     }
68 ]
69 }

```

task 상태 보고 요청 예시

```

1 {
2     "current_status": "ASSIGNED",
3     "status": "RUNNING",
4     "assigned_robot_name": "PICKY1",
5     "result_message": "MOVE_TO_PRODUCT started"
6 }

```

robot 상태 수동 보정 예시

이 API는 admin 검증/디버그용이다.

로봇의 정기 상태 보고는 HTTP가 아니라 ROS2 topic으로 들어온다.

```

1 {
2     "robot_status": "BUSY",
3     "picky_state": "MOVING_TO_PRODUCT",
4     "cobot_state": null,
5     "current_task_id": 101,
6     "battery_level": 87,
7     "pos_x": 1.25,
8     "pos_y": 3.44,
9     "pos_theta": 0.78
10 }

```

선택된 pickup slot 배정 예시

TrafficManager가 PICKUP_ZONE_2를 선택한 경우 TaskManager는 대응되는 PICKUP_SLOT_2를 주문에 배정한다.

```

1 {
2     "pickup_slot_id": 2
3 }

```

호출:

```

1 PATCH /api/fleet/orders/{order_id}

```

6. Fleet Manager 내부 클래스 역할

Fleet Manager는 하나의 ROS2 Node 안에서 일반 Python class들을 조립해 사용한다.

```

1 FleetManagerNode
2   └─ FleetRepository
3   └─ FleetApiServer
4   └─ TaskManager
5   └─ TrafficManager
6   └─ RobotCommandGateway
7   └─ RobotStateMonitor

```

클래스	역할
FleetManagerNode	유일한 ROS2 Node. 하위 클래스 생성, 의존성 연결, timer 등록, callback routing 담당
FleetRepository	DB 접근 단일 계층. just_pick_it_db의 model/service를 사용해 조회/쓰기 수행
FleetApiServer	Fleet Manager 내부 HTTP/WebSocket API 서버. Web Gateway가 호출하는 API 제공
TaskManager	주문/진열 감지, robot_unit 배정, task 생성, 상태 전이, 실패 처리 담당
TrafficManager	zone graph 기반 경로 탐색, 충돌 회피, path 예약/해제, dock 예약 담당
RobotCommandGateway	TaskManager의 task를 ROS2 Action/Service 명령으로 변환해 State Manager에 전송
RobotStateMonitor	PICKY 상태/배터리/pose Topic을 구독하고 DB/TrafficManager에 전달

7. TaskManager ↔ TrafficManager 내부 인터페이스

TaskManager와 TrafficManager는 HTTP/ROS2가 아니라 같은 프로세스 안의 Python method call로 통신한다.

PathResult

```

1 @dataclass(frozen=True)
2 class PathResult:
3     ok: bool
4     waypoints: tuple[str, ...] = ()

```

```

5 cost: float | None = None
6 reason: str | None = None

```

필드	의미
ok	경로 예약 성공 여부
waypoints	시작 zone부터 목적지 zone까지의 zone_name 목록
cost	현재 구현 기준 hop 수
reason	실패 사유

TrafficManager 제공 메서드

메서드	목적
reserve_path(robot_id, task_id, source_zone, target_zone)	목적지가 확정된 이동 경로 예약
reserve_nearest_from(robot_id, task_id, source_zone, candidates)	후보 zone 중 가장 가까운 경로 선택 및 예약
attach_task_id(robot_id, task_id)	task_id=None으로 임시 예약한 path 에 DB task_id 연결
reserve_return_home_path(robot_id, task_id, source_zone)	standby zone 복귀 경로 예약
reserve_dock_path(robot_id, task_id, source_zone)	충전 dock 선택 및 도킹 경로 예약
update_path_progress(robot_id, task_id, current_waypoint_index)	waypoint 통과에 따른 path 점유 해제
release_path(robot_id, task_id)	task 종료 시 path 예약 해제
notify_state(robot_id, state)	RobotStateMonitor가 수신한 PICKY state 전달

상품 후보 선택 호출 예시

```

1 candidates = {
2     "PRODUCT_ZONE_1": 2,
3     "PRODUCT_ZONE_3": 1,

```

```

4 }
5
6 result = traffic_manager.reserve_nearest_from(
7     robot_id="PICKY1",
8     task_id=None,
9     source_zone="STANDBY_ZONE_1",
10    candidates=candidates,
11 )
12
13 if result.ok:
14     selected_zone = result.waypoints[-1]

```

처리 원칙:

```

1 1. TaskManager가 상품 후보 dict를 만든다.
2 2. TrafficManager는 후보 zone 중 가장 가까운 경로를 예약한다.
3 3. TaskManager는 result.waypoints[-1]로 선택된 zone을 확인한다.
4 4. TaskManager가 해당 상품의 MOVE_TO_PRODUCT task를 생성한다.
5 5. DB에서 발행된 task_id를 attach_task_id()로 TrafficManager 예약에 연결한다.
6 6. task 생성 실패 시 release_path(robot_id, None)으로 임시 예약을 해제한다.

```

픽업 zone 선택 호출 예시

```

1 candidates = {
2     "PICKUP_ZONE_1": 1,
3     "PICKUP_ZONE_2": 1,
4 }
5
6 result = traffic_manager.reserve_nearest_from(
7     robot_id="PICKY1",
8     task_id=None,
9     source_zone="PRODUCT_ZONE_3",
10    candidates=candidates,
11 )
12
13 if result.ok:
14     selected_pickup_zone = result.waypoints[-1]

```

처리 원칙:

```

1 1. 모든 상품 상차가 끝난다.
2 2. TaskManager가 EMPTY pickup slot을 FleetRepository에서 조회한다.
3 3. EMPTY slot에 대응되는 pickup zone 후보 dict를 만든다.
4 4. TrafficManager가 가장 가까운 pickup zone을 선택한다.
5 5. TaskManager가 선택된 pickup zone과 같은 번호의 pickup slot을 주문에 배정한다.
6 6. MOVE_TO_PICKUP / INSPECTION / UNLOAD task를 생성한다.

```

8. Web Gateway ↔ LLM 담당 모듈

Web Gateway가 관리자 자연어 명령을 구조화된 action으로 변환하기 위해 LLM 담당 모듈을 호출한다.

LLM 담당 모듈은 DB를 직접 수정하지 않는다.

IF ID	호출 시 점	송신자	수신자	목적	요청 데 이터	응답 데 이터	프로토 콜
-------	-----------	-----	-----	----	------------	------------	----------

IF-L-01	관리자 자연어 명령 입력 시	Admin UI	Web Gateway	자연어 명령 전달	POST /api/admin/llm/messages	LLM 처리 결과	HTTP/ TCP
IF-L-02	자연어 파싱 필요 시	Web Gateway	LLM 담당 모듈	진열 명령 파싱	message, context	action, product, quantity, policy	Python call 또는 HTTP/ TCP
IF-L-03	DISPLAY 파싱 성공 시	Web Gateway	Fleet API	진열 요청 생성	POST /api/admin/display-items	display_item	HTTP/ TCP

LLM action

action	의미
DISPLAY	진열 명령
INVENTORY_SUMMARY	재고 요약 질의
EXCEPTION_SUMMARY	예외 요약 질의
CHAT	일반 응답 또는 해석 불가

IF-L-02 요청 예시

```

1 {
2   "message": "콜라 5개 진열해줘",
3   "context": {
4     "products": [
5       {
6         "product_id": 1,
7         "name": "콜라"
8       },
9       {
10        "product_id": 2,
11        "name": "생수"
12      }
13    ],
14    "low_stock_count": 2,

```

```

15     "unresolved_exception_count": 0
16 }
17 }

```

IF-L-02 응답 예시

```

1 {
2     "result": "ok",
3     "action": "DISPLAY",
4     "product_id": 1,
5     "product_name": "콜라",
6     "requested_quantity": 5,
7     "display_policy": "REQUESTED_QUANTITY",
8     "message": "콜라 5개 진열 요청으로 해석했습니다.",
9     "provider": "llm"
10 }

```

처리 원칙:

1. LLM 담당 모듈은 task를 직접 만들지 않는다.
2. Web Gateway는 LLM 결과가 DISPLAY이면 Fleet API에 display_item 생성을 요청한다.
3. FleetRepository는 display_item을 REQUESTED로 저장한다.
4. TaskManager는 REQUESTED display_item을 polling으로 감지한다.
5. TaskManager가 진열 task 5개를 생성한다.

9. PICKY/COBOT State Manager ↔ AI Server Vision

Vision 요청은 이미지/프레임이 발생하는 로봇 쪽에서 직접 수행한다.

Web Gateway와 Fleet API는 이미지/영상 데이터를 중계하지 않는다.

IF ID	호출 시 점	송신자	수신자	목적	요청 데 이터	응답 데 이터	프로토 콜
IF-V-01	SORTING_AND_LOADING 수행 전	COBOT State Manager	AI Vision Server	주문 상품 인식 및 좌표 추정	task_id, product , camera_frame	detected, coordinates_6d , confidence	HTTP/gRPC/ROS2 Service
IF-V-02	INSPECTION 수행 중	COBOT State Manager	AI Vision Server	주문 상품 검수	task_id, order_items, camera_frame	missing , excess, wrong	HTTP/gRPC/ROS2 Service

IF-V-03	SORTING_LOADD 진행 중	COBOT State Manager	AI Vision Server	창고 상품 인식 및 수량 감지	task_id, display_item, camera_frame	detect_d_quantity, objects	HTTP/gRPC/ROS2 Service
IF-V-04	DISPLAY_SCAN 수행 중	COBOT State Manager	AI Vision Server	진열대 빈자리 탐색	task_id, display_item, camera_frame	empty_position, confidence	HTTP/gRPC/ROS2 Service
IF-V-05	이동/작업 중 사람 감지 필요 시	Vision Node 또는 State Manager	AI Vision Server	사람 감지	robot, camera_frame, zone	person_detected, confidence	HTTP/gRPC/ROS2 Topic/Service

Vision 결과 처리 원칙

항목	원칙
상품 인식 성공	COBOT State Manager가 작업을 진행하고 Fleet Manager에 result 반환
상품 인식 실패	Fleet Manager가 재시도 또는 exception_log 기록
검수 실패	Fleet Manager가 INSPECTION_FAIL 예외 기록
사람 감지	Fleet Manager가 HUMAN_DETECTED 예외 기록 및 필요 시 긴급 정지
이미지 중계	Web Gateway와 Fleet API는 이미지/영상 프레임을 중계하지 않음

10. Fleet Manager ↔ ROS2 / State Manager

Fleet Manager는 RobotCommandGateway를 통해 PICKY/COBOT State Manager에 실제 작업을 지시한다.

State Manager는 로봇 컨트롤러와 통신하고, 결과를 Fleet Manager에 반환한다.

ROS2 인터페이스 분류

용도	방식
PICKY 이동/복귀 명령	ROS2 Action
PICKY 도킹 명령	ROS2 Action
COBOT 선별/상차/검수/하차/진열 명령	ROS2 Action, 인터페이스 확정 대기
로봇 상태 보고	ROS2 Topic
작업 진행 feedback	ROS2 Action feedback
작업 완료/실패 result	ROS2 Action result
긴급 정지/재개	ROS2 Service

Fleet Manager ↔ PICKY State Manager

IF ID	호출 시점	송신자	수신자	목적	ROS2 이름	ROS2 타입	Fleet Manager 처리
IF-R-01	MOVE_TO_PRODUCT 실행 시	Fleet Manager	PICKY State Manager	상품 구역 이동 명령	/picky_ns/move_command	MoveCommand.action	성공 시 task SUCCESS, 실패 시 NAVIGATION_FAILED

IF-R-02	MOVE_TO_PICKUP 실행 시	Fleet Manager	PICKY State Manager	픽업존 이동 명령	/picky_ns/move_command	MoveCommand.action	성공 시 task SUCCESS, 실패 시 NAVIGATION_FAILED
IF-R-03	MOVE_TO_STOCK 실행 시	Fleet Manager	PICKY State Manager	창고 구역 이동 명령	/picky_ns/move_command	MoveCommand.action	성공 시 task SUCCESS, 실패 시 NAVIGATION_FAILED
IF-R-04	MOVE_TO_DISPLAY 실행 시	Fleet Manager	PICKY State Manager	진열 구역 이동 명령	/picky_ns/move_command	MoveCommand.action	성공 시 task SUCCESS, 실패 시 NAVIGATION_FAILED
IF-R-05	RETURN_HOME 실행 시	Fleet Manager	PICKY State Manager	standby zone 복귀	/picky_ns/move_command	MoveCommand.action	성공 시 task SUCCESS
IF-R-06	DOCK_IN 실행 시	Fleet Manager	PICKY State Manager	충전 도크 정밀 진입	/picky_ns/dock_command	DockCommand.action	성공 시 task SUCCESS

IF-R-07	PICKY 상태 변경 시	PICKY State Manager	Fleet Manager	PICKY 상태 보고	/picky_ns/picky_state	std_msgs/String	Traffic Manager 상태 갱신 및 DB 반영
IF-R-08	PICKY 배터리 변경 시	PICKY State Manager	Fleet Manager	배터리 상태 보고	/picky_ns/battery_percent	std_msgs/Float32	battery_level DB 반영
IF-R-09	PICKY pose 변경 시	AMCL	Fleet Manager	위치 보고	/picky_ns/amcl_pose	geometry_msgs/PoseWithCovarianceStamped	pos_x/ pos_y/ pos_theta DB 반영
IF-R-10	긴급 정지/재개 시	Fleet Manager	PICKY State Manager	긴급 정지/해제	/picky_ns/emergency_control	EmergencyControl.srv	Admin emergency/resume 변환

Fleet Manager ↔ COBOT State Manager

IF ID	호출 시점	송신자	수신자	목적	ROS2 이름	ROS2 타입	Fleet Manager 처리
IF-R-11	SORTING_AND_LOADING 실행 시	Fleet Manager	COBOT State Manager	주문/진열 상품 선별 및 PICKY 상차	/cobot_ns/execute_task	ExecuteTask.action 예정	성공 시 task SUCCESS

IF-R-12	INSPECTION 실행 시	Fleet Manager	COBOT State Manager	주문 상 품 검수	/cobot _ns}/execute_task	ExecuteTask.action 예 정	성공 시 order_item.status=INSPECTED
IF-R-13	UNLOADED 실행 시	Fleet Manager	COBOT State Manager	픽업 슬 롯 하차	/cobot _ns}/execute_task	ExecuteTask.action 예 정	성공 시 pickup_slot=OCCUPIED, order=PICKUP_READY
IF-R-14	DISPLAY_SCAN 실행 시	Fleet Manager	COBOT State Manager	진열대 빈자리 탐색	/cobot _ns}/execute_task	ExecuteTask.action 예 정	성공 시 task SUCCESS
IF-R-15	DISPLAY_PLACEMENT 실행 시	Fleet Manager	COBOT State Manager	진열 상 품 배치	/cobot _ns}/execute_task	ExecuteTask.action 예 정	성공 시 display_item=COMPLETED, stock_qty 증가
IF-R-16	COBOT 상태 변 경 시	COBOT State Manager	Fleet Manager	COBOT 상태 보 고	/cobot _ns}/cobot_state	std_msgs/String	robot 상태 반 영

IF-R-17	긴급 정지/재개시	Fleet Manager	COBOT State Manager	긴급 정지/해제	/cobots/emergency_control	EmergencyControl.srv	Admin emergency/resume 변환
---------	-----------	---------------	---------------------	----------	---------------------------	----------------------	---------------------------

주의: 현재 저장소에는 COBOT ExecuteTask.action이 아직 최종 정의되어 있지 않다.
RobotCommandGateway.send_cobot_task()는 인터페이스 확정 전까지 False를 반환하며,
TaskManager는 COBOT task를 ASSIGNED 상태로 유지하고 다음 dispatch cycle에서 재시도한다.

11. ROS2 PICKY Interface

적용 로봇

robot_name	ROS namespace
PICKY1	/picky1
PICKY2	/picky2

담당 task_type

task_type	설명
MOVE_TO_PRODUCT	주문 상품 보관 구역으로 이동
MOVE_TO_PICKUP	픽업존으로 이동
MOVE_TO_STOCK	창고 구역으로 이동
MOVE_TO_DISPLAY	진열 구역으로 이동
RETURN_HOME	standby zone 복귀
DOCK_IN	충전 도크 정밀 진입
CHARGE	충전 상태 진입/유지

PICKY ROS2 Interface

IF ID	호출 시점	송신자	수신자	목적	ROS2 이름	ROS2 타입
-------	-------	-----	-----	----	---------	---------

IF-P-01	이동 task 실행 시	Fleet Manager	PICKY State Manager	waypoint 이동 명령	/picky_ns/move_command	MoveCommand.action
IF-P-02	도킹 task 실행 시	Fleet Manager	PICKY State Manager	충전 도크 정밀 진입	/picky_ns/dock_command	DockCommand.action
IF-P-03	상태 변경 시	PICKY State Manager	Fleet Manager	상태 보고	/picky_ns/picky_state	std_msgs/String
IF-P-04	배터리 변경 시	PICKY State Manager	Fleet Manager	배터리 보고	/picky_ns/battery_percent	std_msgs/Float32
IF-P-05	pose 변경 시	AMCL	Fleet Manager	위치 보고	/picky_ns/amcl_pose	geometry_msgs/PoseWithCovarianceStamped
IF-P-06	긴급 정지/해제 시	Fleet Manager	PICKY State Manager	긴급 정지/해제	/picky_ns/emergency_control	EmergencyControl.srv

MoveCommand.action

```

1  # Goal
2  string task_type
3  geometry_msgs/PoseStamped[] waypoints
4  ---
5  # Result
6  bool success
7  string message
8  ---
9  # Feedback
10 uint8 current_waypoint_index
11 uint8 total_waypoints

```

MoveCommand goal 예시

```
1 {
2   "task_type": "MOVE_TO_PRODUCT",
3   "waypoints": [
4     {
5       "header": {
6         "frame_id": "map"
7       },
8       "pose": {
9         "position": {
10          "x": 1.2,
11          "y": 3.4,
12          "z": 0.0
13        },
14        "orientation": {
15          "z": 0.0,
16          "w": 1.0
17        }
18      }
19    }
20  ]
21 }
```

DockCommand.action

```
1 # Goal
2 int32 task_id
3 string dock_name
4 string start_zone_name
5 ---
6 # Result
7 bool success
8 string message
9 ---
10 # Feedback
11 string phase
12 float32 progress
13 string message
```

처리 원칙

1. RobotCommandGateway가 task_id와 action goal을 내부적으로 매핑한다.
2. MoveCommand.action goal 자체에는 task_id가 없다.
3. action feedback의 current_waypoint_index는 TrafficManager.update_path_progress()에 전달한다.
4. action result success=true이면 task SUCCESS로 처리한다.
5. action result success=false이면 task FAILED 및 exception_log를 기록한다.
6. DOCK_IN은 move_command가 아니라 dock_command를 사용한다.

12. ROS2 COBOT Interface

적용 로봇

robot_name	ROS namespace
COBOT1	/cobot1

COBOT2	/cobot2
--------	---------

담당 task_type

task_type	설명
SORTING_AND_LOAD	주문/진열 상품 선별 및 PICKY 상차
INSPECTION	주문 상품 검수
UNLOAD	픽업 슬롯 하차
DISPLAY_SCAN	진열대 빈자리 탐색
DISPLAY_PLACE	진열 상품 배치

COBOT ROS2 Interface

IF ID	호출 시 점	송신자	수신자	목적	ROS2 이 름	ROS2 타 입
IF-CB-01	COBOT task 실행 시	Fleet Manager	COBOT State Manager	COBOT 작업 명 령	/cobot_ ns}/execute_task	Execute Task.action 예약
IF-CB-02	상태 변경 시	COBOT State Manager	Fleet Manager	COBOT 상태 보 고	/cobot_ ns}/cobot_state	std_msgs/String
IF-CB-03	긴급 정지/해제 시	Fleet Manager	COBOT State Manager	긴급 정지/해제	/cobot_ ns}/emergency_control	EmergencyControl.srv

ExecuteTask.action 초안

현재 저장소에는 아직 최종 ExecuteTask.action이 없다.

아래는 COBOT 담당자와 맞춰야 할 권장 초안이다.

```

1 # Goal
2 int32 task_id
3 int32 order_id
4 int32 order_item_id
5 int32 display_item_id
6 string task_type

```

```

7  int32 source_zone_id
8  int32 target_zone_id
9  int32 product_id
10 string product_name
11 int32 quantity
12 int32 requested_quantity
13 int32 detected_quantity
14 int32 pickup_slot_id
15 ---
16 # Result
17 bool success
18 string status
19 string result_message
20 int32 detected_quantity
21 int32 stock_delta
22 ---
23 # Feedback
24 string status
25 string message
26 float32 progress

```

ExecuteTask goal 예시 - SORTING_AND_LOAD 주문

```

1  {
2    "task_id": 102,
3    "order_id": 7,
4    "order_item_id": 22,
5    "display_item_id": 0,
6    "task_type": "SORTING_AND_LOAD",
7    "source_zone_id": 12,
8    "target_zone_id": 6,
9    "product_id": 3,
10   "product_name": "과자",
11   "quantity": 1,
12   "requested_quantity": 0,
13   "detected_quantity": 0,
14   "pickup_slot_id": 0
15 }

```

ExecuteTask goal 예시 - SORTING_AND_LOAD 진열

```

1  {
2    "task_id": 203,
3    "order_id": 0,
4    "order_item_id": 0,
5    "display_item_id": 1,
6    "task_type": "SORTING_AND_LOAD",
7    "source_zone_id": 4,
8    "target_zone_id": 4,
9    "product_id": 1,
10   "product_name": "콜라",
11   "quantity": 0,
12   "requested_quantity": 5,
13   "detected_quantity": 0,
14   "pickup_slot_id": 0
15 }

```

ExecuteTask goal 예시 - DISPLAY_PLACE

```

1  {
2    "task_id": 206,
3    "order_id": 0,
4    "order_item_id": 0,
5    "display_item_id": 1,

```

```

6  "task_type": "DISPLAY_PLACE",
7  "source_zone_id": 14,
8  "target_zone_id": 14,
9  "product_id": 1,
10 "product_name": "콜라",
11 "quantity": 0,
12 "requested_quantity": 5,
13 "detected_quantity": 5,
14 "pickup_slot_id": 0
15 }

```

처리 원칙

- 1 1. COBOT State Manager는 ExecuteTask.action goal을 받아 작업을 수행한다.
- 2 2. Vision 요청이 필요하면 COBOT State Manager가 AI Vision Server에 직접 요청한다.
- 3 3. 본동작이 끝난 직후 바로 SUCCESS를 반환하지 않는다.
- 4 4. STOWING_ARM 상태 진입을 feedback으로 Fleet Manager에 알려야 한다.
- 5 5. Fleet Manager는 STOWING_ARM feedback을 받으면 다음 PICKY 이동을 선계 hoạch할 수 있다.
- 6 6. 팔 복귀까지 완료된 뒤 action result success=true를 반환한다.
- 7 7. action result success=false이면 Fleet Manager가 task FAILED 및 exception_log를 기록한다.
- 8 8. DISPLAY_PLACE result의 detected_quantity, stock_delta는 진열 완료 처리에 사용한다.

13. Emergency Stop Service

항목	값
Service name	/robot_namespace/emergency_control
Type	just_pick_it_interfaces/srv/EmergencyControl
emergency_stop=true	긴급 정지
emergency_stop=false	긴급 정지 해제 요청

EmergencyControl.srv

```

1 bool emergency_stop
2 string reason
3 int32 task_id
4 string request_id
5 ---
6 bool accepted
7 string status
8 string message

```

처리 원칙

- 1 1. Admin UI가 emergency-stop/resume 요청을 Web Gateway에 보낸다.
- 2 2. Web Gateway가 Fleet API로 요청을 전달한다.
- 3 3. FleetManagerNode가 DB 상태 전이를 수행한다.

- 4 4. RobotCommandGateway가 각 State Manager의 emergency_control service를 호출한다.
- 5 5. 실패한 로봇은 exception_log 또는 로그에 기록한다.

14. Robot ID / Namespace

robot_id	robot_name	robot_type	robot_unit	ROS namespace	미니맵 표시
1	PICKY1	PICKY	PICKY_UNIT_1	/picky1	표시
2	COBOT1	COBOT	PICKY_UNIT_1	/cobot1	표시 안 함
3	PICKY2	PICKY	PICKY_UNIT_2	/picky2	표시
4	COBOT2	COBOT	PICKY_UNIT_2	/cobot2	표시 안 함

15. Zone / Slot 기준

구분	의미
ZONE	PICKY가 이동하거나 정차하는 위치
SLOT	COBOT이 물건을 집거나 내려놓는 실제 작업 위치

주요 매핑

PICKY 정차 위치	COBOT 작업 위치
PRODUCT_ZONE_1	PRODUCT_SLOT_1
PRODUCT_ZONE_2	PRODUCT_SLOT_2
PRODUCT_ZONE_3	PRODUCT_SLOT_3
PRODUCT_ZONE_4	PRODUCT_SLOT_4

PRODUCT_ZONE_5	PRODUCT_SLOT_5
PRODUCT_ZONE_6	PRODUCT_SLOT_6
PICKUP_ZONE_1	PICKUP_SLOT_1
PICKUP_ZONE_2	PICKUP_SLOT_2
STOCK_ZONE	STOCK_SLOT

처리 원칙

1. PICKY 이동 task의 target은 ZONE을 사용한다.
2. COBOT 작업 task의 source/target은 SLOT 또는 작업 기준 위치를 사용한다.
3. 주문 상품 보관 위치는 PRODUCT_SLOT_*이다.
4. PICKY가 해당 상품 앞에 정차하는 위치는 PRODUCT_ZONE_*이다.
5. 진열 시 PICKY는 STOCK_ZONE에서 상품을 싣고 PRODUCT_ZONE_*으로 이동한다.
6. 진열 시 COBOT은 STOCK_SLOT에서 상품을 싣고 PRODUCT_SLOT_*에 진열한다.
7. pickup slot은 상품 상차가 모두 끝난 뒤 MOVE_TO_PICKUP 직전에 배정한다.

16. 주문 task 흐름

주문 task는 상품별 이동/상차를 반복한 뒤 픽업존 이동/검수/하차로 마무리한다.

```

1 상품 1:
2   MOVE_TO_PRODUCT
3   SORTING_AND_LOAD
4
5 상품 2:
6   MOVE_TO_PRODUCT
7   SORTING_AND_LOAD
8
9 ...
10
11 모든 상품 상차 완료 후:
12   MOVE_TO_PICKUP
13   INSPECTION
14   UNLOAD

```

처리 원칙

1. TaskManager가 주문 item 목록을 dict로 관리한다.
2. TaskManager가 남은 상품 후보를 {PRODUCT_ZONE_N: quantity} 형태로 TrafficManager에 전달한다.
3. TrafficManager가 가장 가까운 상품 zone을 선택하고 path를 예약한다.
4. TaskManager가 선택된 상품에 대해 MOVE_TO_PRODUCT/SORTING_AND_LOAD task를 생성한다.
5. 모든 상품 상차가 끝나면 EMPTY pickup slot을 조회한다.
6. TaskManager가 pickup zone 후보를 TrafficManager에 전달한다.
7. TrafficManager가 가장 가까운 pickup zone을 선택한다.
8. TaskManager가 같은 번호의 pickup slot을 주문에 배정한다.
9. MOVE_TO_PICKUP/INSPECTION/UNLOAD task를 생성한다.
10. UNLOAD SUCCESS 시 order=PICKUP_READY, pickup_slot=OCCUPIED로 전이한다.

17. 진열 task 흐름

진열 task는 LLM/Web Gateway가 만든 display_item을 기준으로 TaskManager가 생성한다.

```
1 MOVE_TO_STOCK
2 SORTING_AND_LOAD
3 MOVE_TO_DISPLAY
4 DISPLAY_SCAN
5 DISPLAY_PLACE
```

처리 원칙

```
1 1. LLM 담당 모듈은 자연어를 해석해 DISPLAY action을 반환한다.
2 2. Web Gateway는 DISPLAY 결과를 Fleet API에 전달해 display_item을 생성한다.
3 3. Fleet Manager는 REQUESTED display_item을 polling으로 감지한다.
4 4. TaskManager가 robot_unit을 배정한다.
5 5. TaskManager가 진열 task 5개를 생성한다.
6 6. DISPLAY_PLACE SUCCESS 후 product.stock_qty와 display_item 상태를 반영한다.
```

18. 작업 종료 후 복귀/도킹/충전 흐름

주문 또는 진열 작업이 끝난 뒤 TaskManager가 다음 작업 여부와 배터리 상태를 보고 복귀/도킹/충전을 결정한다.

```
1 UNLOAD 또는 DISPLAY_PLACE 완료
2 -> 대기 주문/진열이 있으면 다음 작업 수행
3 -> 대기 작업이 없으면 RETURN_HOME
4 -> standby zone 도착
5 -> 필요하면 DOCK_IN
6 -> 도킹 성공 후 CHARGE 또는 robot_status=CHARGING
```

task_type	의미	TrafficManager 호출
RETURN_HOME	standby zone 복귀	reserve_return_home_path()
DOCK_IN	충전 도크 정밀 진입	reserve_dock_path()
CHARGE	충전 상태 진입/유지	경로 예약 없음

처리 원칙

```
1 1. RETURN_HOME은 standby zone 도착까지만 담당한다.
2 2. DOCK_IN은 충전 dock 선택과 정밀 도킹을 담당한다.
3 3. CHARGE는 충전 상태 진입/유지로 본다.
4 4. 배터리가 기준치 이상으로 회복되면 TaskManager가 CHARGE SUCCESS를 처리한다.
```

19. ENUM 요약

order_status

1	ORDER_RECEIVED
2	ORDER_WAIT
3	SORTING
4	DELIVERING
5	INSPECTING
6	PICKUP_READY
7	COMPLETED
8	ERROR

order_item_status

1	WAITING
2	SORTED
3	INSPECTED
4	MISSING
5	EXCESS
6	MISMATCH

pickup_slot_status

1	EMPTY
2	RESERVED
3	OCCUPIED
4	BLOCKED

robot_type

1	PICKY
2	COBOT

robot_status

1	OFFLINE
2	IDLE
3	BUSY
4	CHARGING
5	EMERGENCY_STOP
6	ERROR

picky_state

1	CHARGING
2	STANDBY
3	MOVING_TO_PRODUCT
4	WAITING_FOR_COBOT
5	MOVING_TO_PICKUP
6	MOVING_TO_STOCK
7	MOVING_TO_DISPLAY
8	RETURNING
9	DOCKING
10	ERROR_RECOVERY

cobot_state

1	STANDBY
2	SORTING
3	LOADING
4	INSPECTING
5	UNLOADING
6	SCANNING
7	PLACING
8	STOWING_ARM

9 SAFETY_STOPPED

task_type

1 MOVE_TO_PRODUCT
2 SORTING_AND_LOAD
3 MOVE_TO_PICKUP
4 INSPECTION
5 UNLOAD
6 MOVE_TO_STOCK
7 MOVE_TO_DISPLAY
8 DISPLAY_SCAN
9 DISPLAY_PLACE
10 RETURN_HOME
11 DOCK_IN
12 CHARGE

task_status

1 QUEUED
2 ASSIGNED
3 RUNNING
4 PAUSED
5 SUCCESS
6 FAILED
7 CANCELLED

display_policy

1 REQUESTED_QUANTITY
2 ALL_DETECTED

display_item_status

1 REQUESTED
2 ASSIGNED
3 IN_PROGRESS
4 COMPLETED
5 FAILED
6 CANCELLED

exception_type

1 OBSTACLE_DETECTED
2 LOW_BATTERY
3 NAVIGATION_FAILED
4 HARDWARE_ERROR
5 TIMEOUT
6 SORTING_FAIL
7 INSPECTION_FAIL
8 HUMAN_DETECTED
9 SYSTEM_ERROR

llm_action

1 DISPLAY
2 INVENTORY_SUMMARY
3 EXCEPTION_SUMMARY
4 CHAT

20. 금지/주의 항목

항목	이유
UI가 DB에 직접 접근	DB 소유권은 Fleet Manager/FleetRepository에 있다.
Web Gateway가 DB에 직접 접근	Web Gateway는 화면과 프록시만 담당한다.
PICKY/COBOT이 Fleet API task를 직접 조회	task 생성/배정/상태 전이는 Fleet Manager 책임이다.
Web Gateway가 직접 PICKY/COBOT에 이동/작업 명령 전송	로봇 제어 책임은 Fleet Manager와 State Manager가 담당한다.
Web Gateway/Fleet API가 이미지/영상 중계	이미지/영상은 Vision 요청 주체가 AI Vision Server에 직접 보낸다.
주문 생성 시 바로 task 생성	task 생성은 TaskManager 책임이다.
주문/진열 감지를 push 필수로 의존	현재 기준은 TaskManager polling이다.
task_type=PATROL	순찰 시나리오 제거로 사용하지 않는다.
exception_type=FIRE_DETECTED	순찰/화재 감지 시나리오 제거로 사용하지 않는다.
task_type=DELIVERY	운반 흐름은 MOVE_TO_PRODUCT, MOVE_TO_PICKUP 등으로 분리한다.
task_type=PICK_AND_LOAD	최신 명칭은 SORTING_AND_LOAD이다.
task_type=STOCKING_PICK	최신 진열 흐름에서는 사용하지 않는다. SORTING_AND_LOAD를 사용한다.
task_type=MOVE_TO_STORAGE	최신 진열 흐름에서는 사용하지 않는다. MOVE_TO_DISPLAY를 사용한다.

task_type=STOCKING_PLACE	최신 진열 흐름에서는 사용하지 않는다. DISPLAY_PLACE를 사용한다.
task.payload_json 사용	진열 데이터는 display_item, 주문 상품 데이터는 order_item으로 관리한다.
robot_status에 세부 작업 상태를 모두 넣기	큰 상태는 robot_status, 세부 상태는 picky_state/cobot_state로 분리한다.
COBOT을 미니맵 이동 객체로 표시	미니맵에는 robot_type=PICKY만 표시한다.
진열 수량을 단일 quantity만으로 관리	requested_quantity, detected_quantity, display_policy, stock_delta로 구분한다.
pickup slot을 주문 초기에 고정 배정	모든 상품 상차 후 MOVE_TO_PICKUP 직전에 TrafficManager가 선택한 pickup zone에 맞춰 배정한다.
PATCH /api/fleet/robots를 로봇 정기 보고로 사용	로봇 정기 상태는 ROS2 topic으로 보고한다. 해당 API는 admin 수동 보정/검증용이다.